Generative Modelling Case Study: GANs for Synthetic Data Generation

Name: Bhanu Prakash Mallam
ID: 24095576
Github: https://github.com/bhanu7959/Generative-modelling-case-study

---

1. Introduction

A Generative Adversarial Network learns to mimic an unknown distribution of data by training two neural networks competitively against each other. The generator network transforms noise into samples, while the discriminator tries to tell the samples apart from actual data (Goodfellow et al., 2014). Training GANs is notoriously hard due to the simultaneous optimisation of two conflicting networks, and figuring out what makes a certain architecture work is just as critical as getting its loss figure right.

In this document, I provide a case study in which Generative Adversarial Networks (GANs) are built from scratch and used to solve three real-world applications such as the generation of synthetic OCT retinal images, generation of synthetic feature vectors from network traffic for cyber-security research, and generation of hand drawn sketches. In Part 1, I use a two-dimensional toy distribution (a noisy sine curve and a spiral) to develop understanding regarding architectural design choices and then in Part 2 apply the same concepts at scale using images and tabular data.

---

2. Methodology Overview

2.1 The GAN training objective

Standard non-saturating loss was utilized for the GAN along with BCEWithLogitsLoss and Adam optimizer ($\beta = 0.5, 0.999$). One sided label smoothing (real label = 0.9) was employed as well to prevent the discriminator from being too confident (Radford, Metz and Chintala, 2016).

2.2 Why these architectures were chosen

Full-connected neural nets were employed in toy 2D data sets. The original GAN had small hidden layers (32 neurons) with ReLU/LeakyReLU, whereas the modified version deepened the network, applied batch normalization, changed ReLU to GELU, and utilized dropout in the discriminator network. The modifications helped the model learn the more complex spiral distribution and stabilize the training process.

For image generation (Parts 2.1 and 2.3), a DCGAN was implemented where the generator increases the size of the latent vector by using four transposed convolutional layers (1×1 →

4×4 → 8×8 → 16×16 → 32×32) that use batch normalization and ReLU, and then outputs the result using Tanh that matches the [-1,1] normalization of input images. Discriminator is similar but uses striped convolution and LeakyReLU instead. Since convolutions naturally take advantage of spatial locality and weight sharing, which fully connected layers can not do efficiently at this resolution, convolutional layers are the proper way to go when working with image data.

In the case of CICIDS network traffic features (Part 2.2), the data is tabular data and not spatial data, and hence fully connected networks for both the generator and discriminator (similar to Part 1 but scaled up to 68 input features) were used, with batch normalization and dropout playing similar stabilization roles as outlined above.

2.3 Evaluation strategy

The loss values of GANs themselves cannot be considered a reliable measure of sample quality because a balanced loss curve can still go together with mode dropping or blurring effects. For this reason, each experiment comes with its respective quantitative metric that is appropriate for the specific task: mean nearest real neighbor distance in case of the 2-D toy problem, Fréchet Inception Distance for images, and matrix correlation comparison along with PCA projection for tabular traffic data.

---

3. Part 1 — Two-Dimensional GANs

3.1 Sine-wave distribution

A noisy sine wave, $y = \sin(x) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 0.08^2)$, was used as the simplest test case (Figure 1). The original architecture was trained for 600 epochs on 12,000 standardised samples.

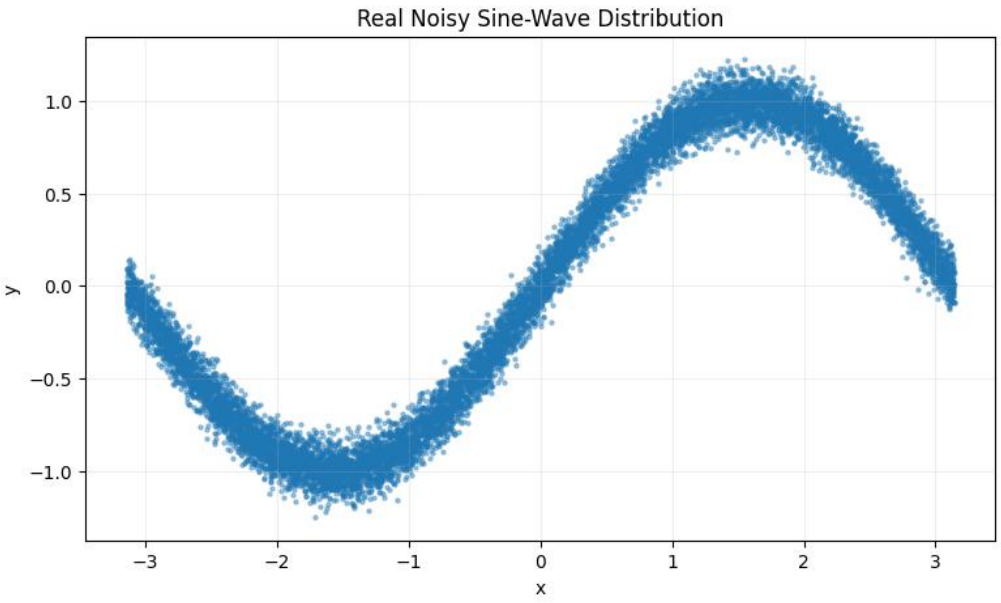

Figure 1. Real noisy sine-wave distribution used to train the baseline GAN.

Generator and discriminator losses reach steady state (generator ≈ 0.80, discriminator ≈ 1.38) following an initial period of instability in the first few hundred epochs (see Figure 2). From an intuitive standpoint, the sample generation closely mimics the curvature and noise distribution of the true data distribution (see Figure 3). It can be inferred from this that a relatively simple fully connected network suffices for this problem.

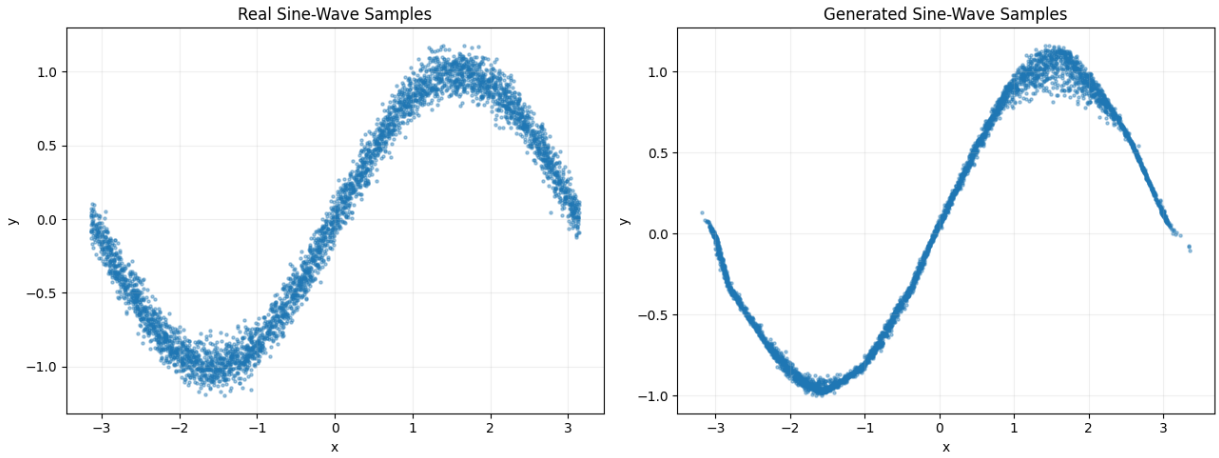

Figure 2. Real (left) versus generated (right) sine-wave samples after 600 epochs.

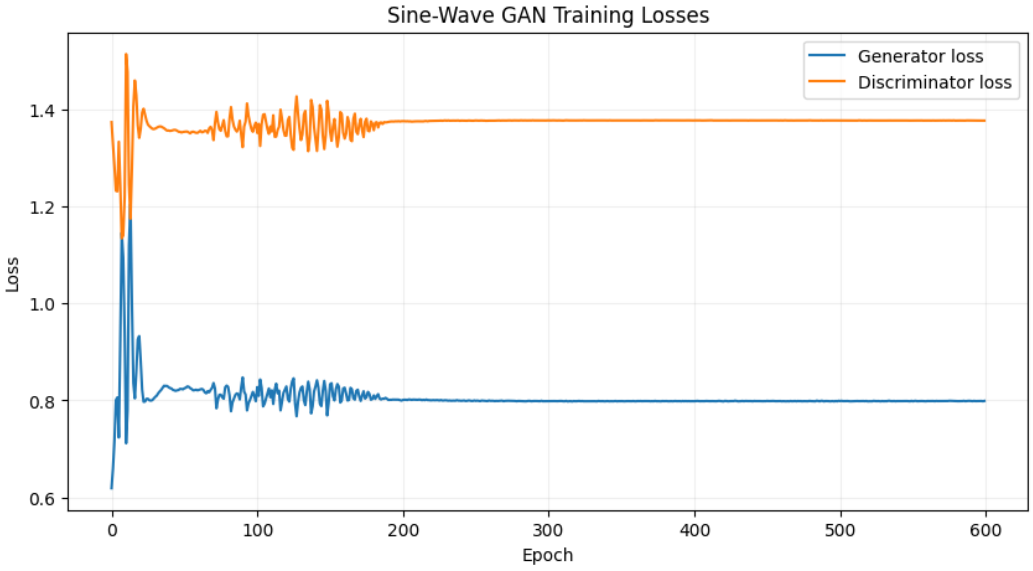

Figure 3. Generator and discriminator training losses for the sine-wave GAN. Losses stabilise after roughly 200 epochs, with no sign of one network overpowering the other.

3.2 Spiral distribution and architecture comparison

Spiral distribution (Figure 4) is a significantly more challenging distribution for generation; this is a curve that twists back onto itself twice-and-a-half, so the generator needs to learn a very complex non-linear manifold that intersects itself in projection.

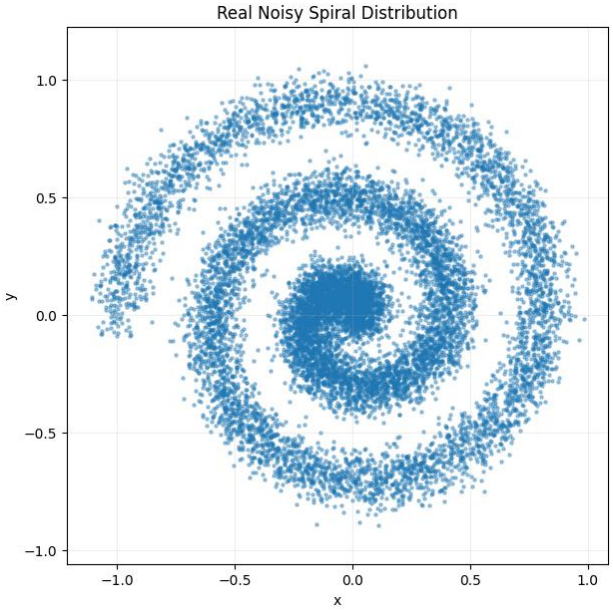

Figure 4. Real noisy spiral distribution.

Training the original model on the spiral results in samples that only partly follow the arms of the spiral but rather cluster at the center, mode collapse of sorts (Figure 5, middle panel). The modified version of the model (with increased depth, GELU activation function, batch normalisation and dropout) generates samples which match the original spiral much better (Figure 5, right panel).

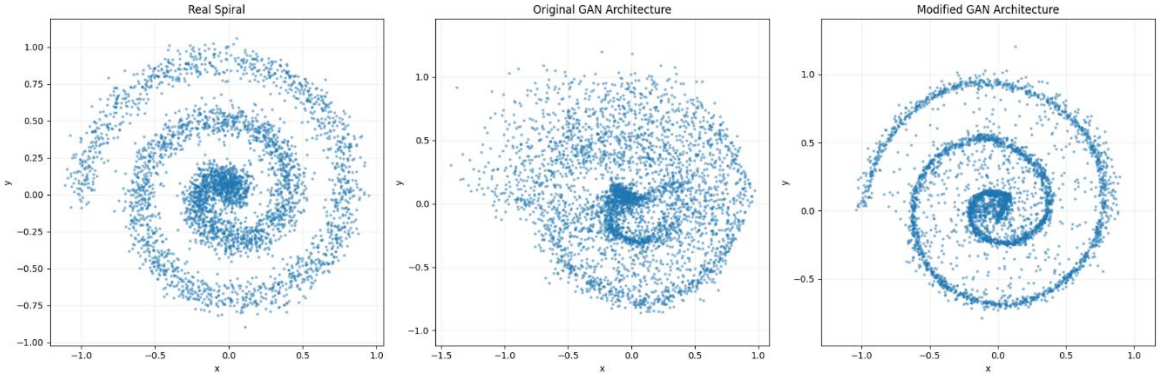

Figure 5. Real spiral (left) versus samples from the original architecture (centre) and the modified, higher-capacity architecture (right).

This qualitative impression is confirmed quantitatively by the mean nearest-real-neighbour distance, which nearly halves between architectures:

Architecture	Mean nearest-real distance
Original GAN	0.0225
Modified GAN	0.0124

From the training losses (Figure 6), one can tell that the modified architecture achieves an equilibrium close to the original one but with much reduced fluctuations when training stabilizes. This finding supports the assumption that batch normalization makes the optimization landscape smoother. It should be noted that neither of the two architectures demonstrates a collapsing discriminator loss.

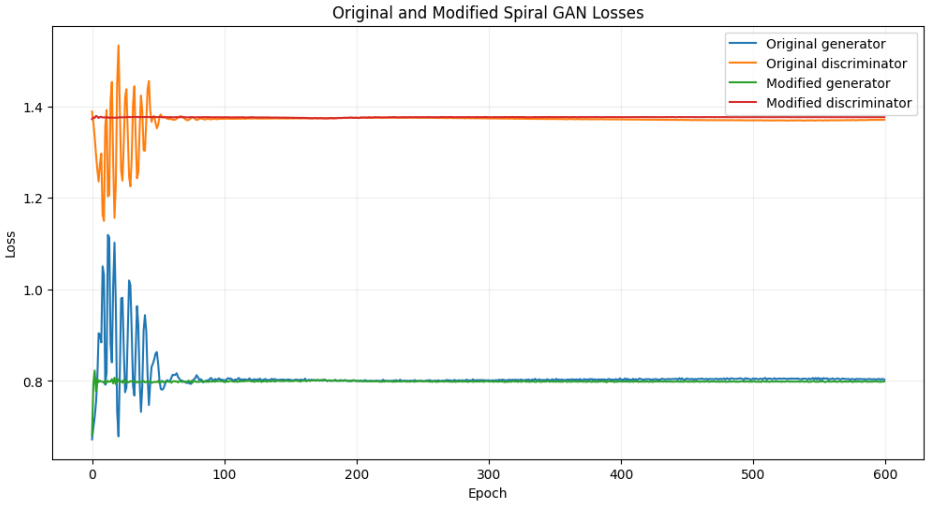

Figure 6. Training losses for the original and modified spiral GANs, plotted together.

Interpretation. Part 1 shows that loss curves indicate training stability, not sample quality. Although both models achieved similar losses, only the modified GAN accurately learned the spiral distribution, highlighting the need for additional evaluation metrics beyond loss curves

---

4. Part 2 — Applied Generative Modelling

4.1 OCTMNIST retinal image synthesis (DCGAN)

The OCTMNIST subset in MedMNIST (Yang et al., 2023) includes 109,309 OCT retina images belonging to four different classes. The images were rescaled to 32x32 and then merged for training the GAN. Since the dataset is unbalanced (see Figure 7 below), the GAN

is biased towards generating images of majority classes.

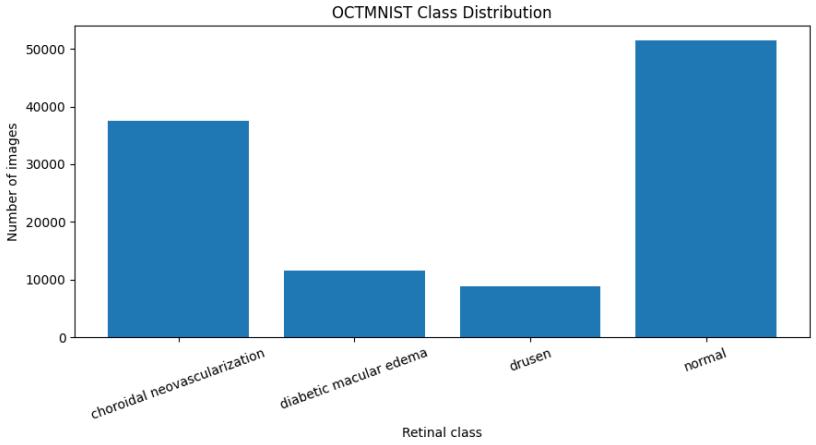

Figure 7. OCTMNIST class distribution across the combined train/validation/test splits.

The DCGAN described in Section 2.2 was trained for 80 epochs with a 100-dimensional latent vector. Figure 8 shows example real OCT images used for training.

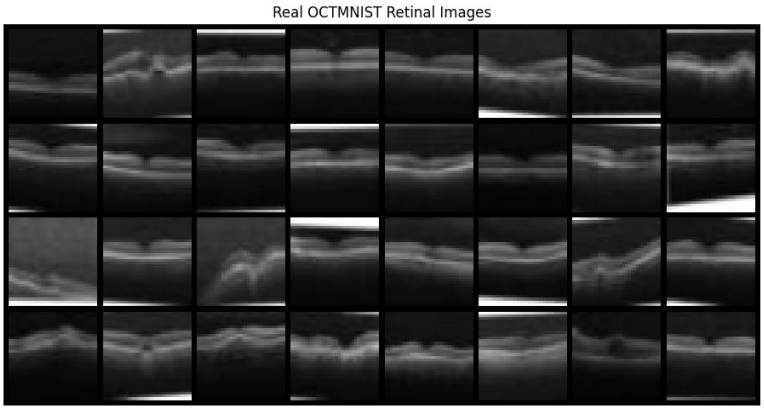

Figure 8. A batch of real OCTMNIST training images.

The Figure 9 reveals an increase in the generator loss with decreasing discriminator loss. Yet, Figure 10 demonstrates a gradual improvement in the generated images, making them sharper and more realistic at epoch 80. This emphasizes the fact that the GAN performance needs to be assessed based on the loss curves and generated images.

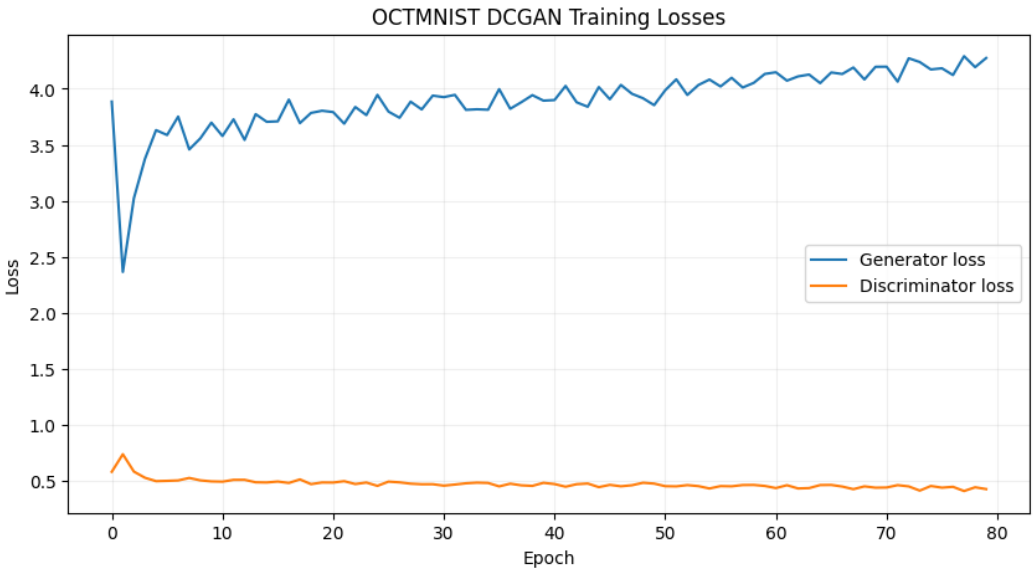

Figure 9. OCTMNIST DCGAN training losses over 80 epochs.

Generator output from the same fixed noise vectors at epochs 1, 20, 40, and 80, showing progressive sharpening of retinal-layer structure.

The produced images (Figure 10) were found to be very similar to the actual OCT scans in terms of the image brightness and layering. The approximate FID score of 0.038 showed a decent similarity of images, even though it could be considered only relative since it was calculated using the light Inception model based on natural images.

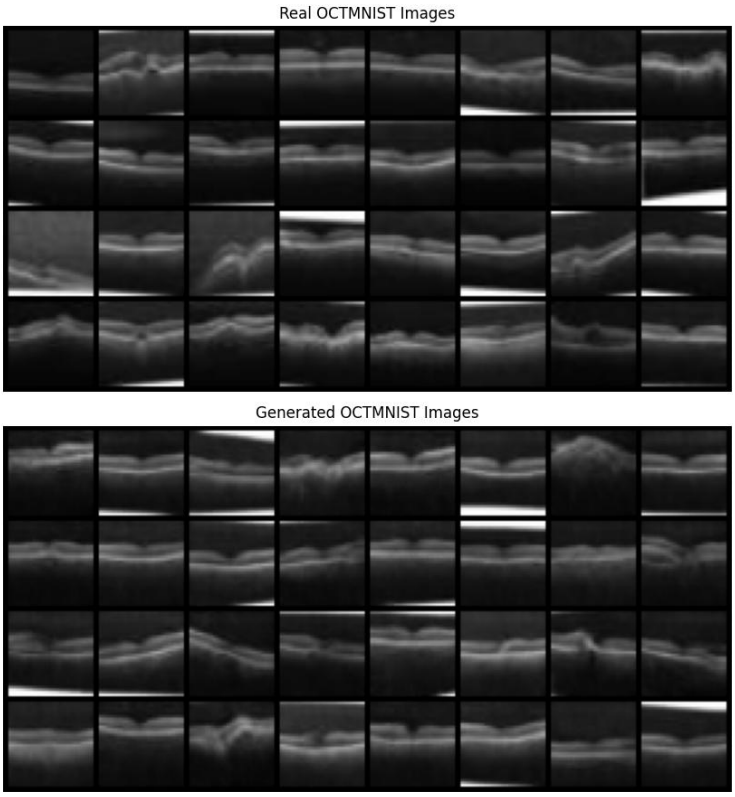

Figure 10. Real (top) versus final generated (bottom) OCTMNIST images.

4.2 CICIDS 2017 synthetic network traffic (tabular GAN)

The second case study made use of the CICIDS2017 DDoS dataset (Sharafaldin, Lashkari and Ghorbani, 2018). A balanced subset of normal and attacks was selected from the dataset due to its highly unbalanced nature (Figure 11).

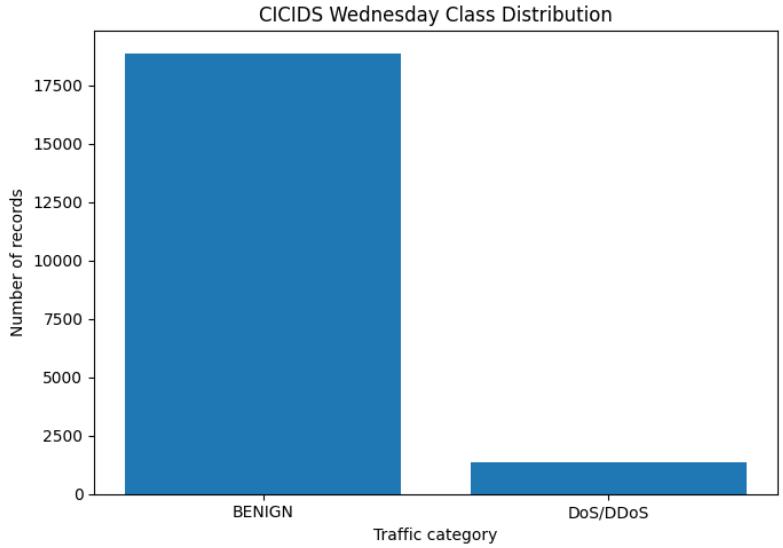

Figure 11. Class distribution of the selected CICIDS Wednesday-style traffic records prior to balanced sampling.

Sixty-eight numeric flow-based features were pre-processed, clipped to avoid any outliers and normalized in the range [-1, 1]. The clipping was done to avoid any big values influencing the normalization procedure.

A fully connected generator/discriminator pair (Section 2.2) was trained for 300 epochs. The loss curves (Figure 12) show the generator loss climbing gradually from ~0.74 to ~1.13 while the discriminator loss falls from ~1.32 to ~1.21 — a slow, steady divergence rather than the abrupt collapse that would signal discriminator dominance, suggesting a reasonably balanced adversarial game over the full run.

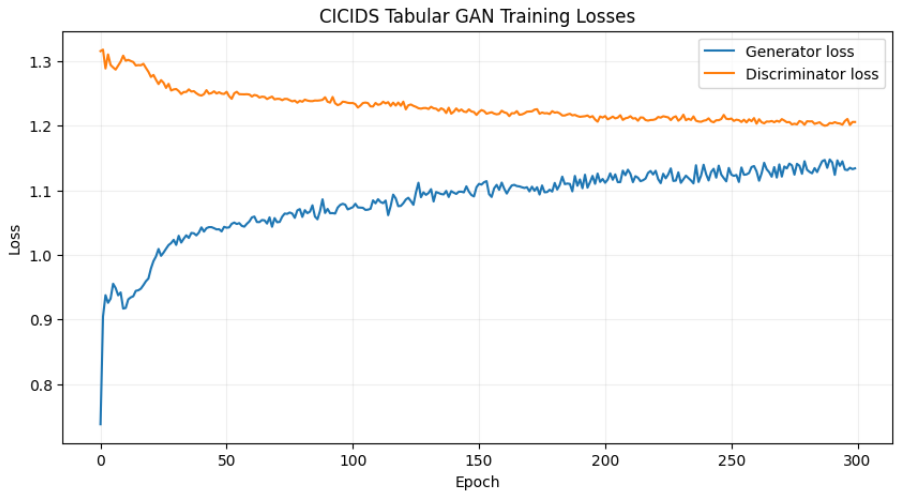

Figure 12. Tabular GAN training losses for the CICIDS traffic generator.

To visualize the 68 dimensional data as 2 dimensional data, PCA method is applied (Figure 13). The produced samples were highly overlapping with the real data but slightly concentrated in low density areas, a common drawback in GANs.

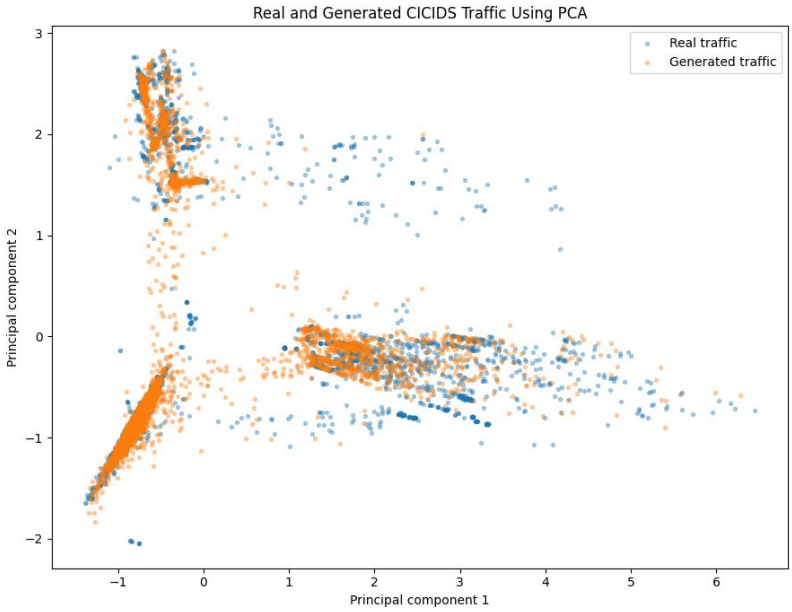

Figure 13. Two-dimensional PCA projection comparing real and generated CICIDS traffic feature vectors.

The largest differences between real and generated data occurred in heavy-tailed timing features. However, the mean correlation difference was only **0.0642**, indicating that the GAN preserved most relationships between features despite some loss of detail.

4.3 QuickDraw birthday-cake sketch generation (DCGAN)

The last application involved applying the OCTMNIST DCGAN model to create 50,000 images of birthday cakes from the QuickDraw database (Ha and Eck, 2018). The created images were sized at 32 × 32 and trained for 100 epochs.

Training losses (Figure 14) followed a similar pattern to the OCTMNIST experiment, with generator loss increasing and discriminator loss stabilizing. Despite this, the generated sketches (Figure 15) successfully captured the main features of birthday cakes, including layered shapes and candle-like strokes, closely resembling the real drawings.

Figure 14. QuickDraw birthday-cake DCGAN training losses.

Figure 15. Real (top) versus generated (bottom) QuickDraw birthday-cake sketches after 100 epochs.

The FID score for QuickDraw was estimated to be 3.245, which is more compared to OCTMNIST because of the varied drawings created by hands. The generated images were also bright and smooth compared to the actual ones; however, they captured the sketching style of the drawings.

A DCGAN can also be used for duck images; however, since there is more variation in the pose, viewpoints, and backgrounds of ducks, it is possible that a larger RGB model will have to be used.

---

5. Discussion and Limitations

Across all five experiments, three patterns recur consistently:

1. **Loss curves diagnose stability, not fidelity.** Across all experiments, the loss curves stabilised before the generated samples reached their best quality, showing that sample-based evaluation was essential alongside loss monitoring.
2. **Architectural capacity trades off against training stability.** The Part 1 spiral experiment showed that deeper networks with batch normalization and improved activation functions learned the distribution more accurately, although they required greater training and tuning effort.
3. **No single metric is sufficient.** Different evaluation metrics measure different aspects of GAN performance. Nearest-neighbour distance, FID, PCA, and correlation analysis each have limitations, so multiple metrics are needed for a reliable assessment of generated data quality.

Overall, the datasets had several limitations: class imbalance in OCTMNIST, difficulty modelling heavy-tailed CICIDS features, and the absence of downstream validation, such as clinical assessment or intrusion-detection testing, before practical use of the generated data

---

6. Conclusion

This case study built GANs from first principles on controlled 2-D distributions, established that architectural capacity and regularisation materially affect distributional coverage on the harder spiral task, and then carried those lessons into three applied domains — medical imaging, network security, and creative sketch generation — using architecture choices (convolutional DCGAN for images, fully connected GAN with batch normalisation and dropout for tabular data) matched to the structure of each dataset. In every case, generated samples visually and statistically approximated the real distribution reasonably well, while quantitative diagnostics (nearest-neighbour distance, FID, correlation comparison) consistently revealed that loss curves alone would have given an incomplete or even misleading picture of model quality. Future work would benefit from conditional generation to address class imbalance in OCTMNIST, higher-fidelity FID computation for cross-domain comparability, and task-based downstream validation for both the medical and cybersecurity applications before any synthetic data generated by these models is used beyond exploratory research.

References

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial nets', *Advances in Neural Information Processing Systems*, 27.

Ha, D. and Eck, D. (2018) 'A neural representation of sketch drawings', *International Conference on Learning Representations (ICLR)*.

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B. and Hochreiter, S. (2017) 'GANs trained by a two time-scale update rule converge to a local Nash equilibrium', *Advances in Neural Information Processing Systems*, 30.

Kingma, D.P. and Ba, J. (2015) 'Adam: A method for stochastic optimization', *International Conference on Learning Representations (ICLR)*.

Radford, A., Metz, L. and Chintala, S. (2016) 'Unsupervised representation learning with deep convolutional generative adversarial networks', *International Conference on Learning Representations (ICLR)*.

Sharafaldin, I., Lashkari, A.H. and Ghorbani, A.A. (2018) 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116.

Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H. and Ni, B. (2023) 'MedMNIST v2 — A large-scale lightweight benchmark for 2D and 3D biomedical image classification', *Scientific Data*, 10, p. 41.